МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
ІМЕНІ ІГОРЯ СІКОРСЬКОГО»

Кафедра прикладної математики

ЗВІТ ЛАБОРАТОРНОЇ РОБОТИ №1

з дисципліни «Бази даних»

на тему: «Побудова B+ дерева»

Виконав:
студент III курсу групи КМ-32,
спеціальність F1
Прикладна математика
Мурсюкаєв Дамір Андрійович

Київ – 2025

ЗМІСТ

1. Правила переведення слова в число..... 3
1.1 Базова функція name_to_key()..... 3
1.2 Комбінована функція name_to_key_full()..... 4
2. Опис структури дерева з прикладами коду..... 5
2.1 Клас BPlusNode – вузол дерева..... 5
2.2 Клас BPlusTree – параметри..... 6
3. Приклади розміщення хеш-кодів у B+ дереві.....7
3.1 Хеш-коди всіх записів довідника..... 7
3.2 Код для виведення структури дерева..... 8
3.3 Структура розміщення хеш-кодів у B+-дереві.....9
4. Схеми пошуку значень в дереві.....10
4.1 Точний пошук – search()..... 10
4.2 Пошук з кешем листка..... 11
4.3 Порівняння операцій пошуку..... 12
4.4 Порівняння операцій вставки та видалення..... 12
Додаток А. Код програми.....13

1. Правила переведення слова в число

Хеш-функція перетворює рядкове ім’я у ціле число. Реалізовано два варіанти – базовий (order-preserving) та комбінований.

1.1 Базова функція name_to_key()
Алгоритм: беремо перші KEY_CHARS=4 символи, кожну літеру перетворюємо у число (a=1…z=26), складаємо поліном з основою BASE=27 (використовуємо “0” для доповнення, якщо ім’я коротше 4 символів).

```
def name_to_key(name: str) -> int:
    chars = name.lower()[:KEY_CHARS]
    key = 0
    for ch in chars:
        if 'a' <= ch <= 'z':
            code = ord(ch) - ord('a') + 1
        else:
            code = 0
        key = key * BASE + code

    padding = KEY_CHARS - len(chars)
    key *= BASE ** padding

    return key
```

Приклад:
Покрокове обчислення для 'Boris':
chars = 'bori' (перші 4 символи)
b=2, o=15, r=18, i=9
key = 0 * 27 + 2 = 2
key = 2 * 27 + 15 = 69
key = 69 * 27 + 18 = 1881
key = 1881 * 27 + 9 = 50796
name_to_key('Boris') = 50 796

1.2 Комбінована функція name_to_key_full()
Якщо два імені починаються однаково (Anna та Annabelle), перші 4 символи збігаються. Рішення: до prefix додається rolling hash решти символів (суфікс), обмежений за модулем щоб не порушити алфавітний порядок.

```
def name_to_key_full(name: str) -> int:
    prefix = name_to_key(name)

    suffix_chars = name.lower()[KEY_CHARS:]
    rolling = 0
    for ch in suffix_chars:
        rolling = rolling * 31 + ord(ch)

    SUFFIX_MOD = BASE ** KEY_CHARS
    return prefix * SUFFIX_MOD + (rolling % SUFFIX_MOD)
```

У цьому коді SUFFIX_MOD задається автоматично через параметри BASE і KEY_CHARS, які можна змінювати в залежності від мови. Його використання є обов’язковим для того, щоб зберегти алфавітний порядок без перекриття значень.

Приклад:
Покрокове обчислення для 'Boris':
name_to_key('Boris') = 50 796
suffix_chars = 's'
rolling = 0 * 31 + 115 = 115
SUFFIX_MOD = 27 ^ 4 = 531 441
prefix = 50 796 * 531 441 + 115 = 26 995 077 151

2. Опис структури дерева з прикладами коду

B+ дерево з параметрами order = 4, max depth = 3 будується на основі двох класів.

2.1 Клас BPlusNode – вузол дерева

```
class BPlusNode:
    def __init__(self, is_leaf=False):
        self.is_leaf = is_leaf
        self.keys = []
        self.children = []
        self.parent = None
        self.next = None

    def __repr__(self):
        return f"{'Leaf' if self.is_leaf else 'Node'}{self.keys}"
```

У табл.1 наведено інформацію, яка міститься у вузлі, в залежності від того, чи є він листком.

Таблиця 1 – Тип вузлів та їх інформація

Тип вузла	keys містить	children містить	next
Внутрішній (is_leaf=False)	Числові роздільники (навігаційні ключі)	Дочірні BPlusNode (вузли нижнього рівня)	None
Листок (is_leaf=True)	Числові хеш-ключі записів	Кортежі (ім’я, телефон)	Наступний листок

2.2 Клас BPlusTree – параметри

```
class BPlusTree:
   def __init__(self, order=4, max_depth=3, use_full_hash=True):
        self.root          = BPlusNode(is_leaf=True)
        self.order         = order
        self.min_keys      = (order + 1) // 2
        self.max_depth     = max_depth
        self._depth        = 1
        self._hash_fn      = name_to_key_full if use_full_hash else
name_to_key

        max_capacity = (order - 1) * (order ** (max_depth - 1))
        print(f"\n[BPlusTree] order={order}, max_depth={max_depth}, "
              f"min_keys={self.min_keys}, max_capacity≈{max_capacity}
records")

        self._cached_leaf  = None
        self._cache_hits   = 0
        self._cache_misses = 0
```

Параметри та їх опис для створення самого дерева наведені у табл. 2.

Таблиця 2 – Параметри побудови дерева

Параметр	Значення	Опис
order	4	Вузол розбивається коли досягає 4 ключів
min_keys	⌈4/2⌉ = 2	Мінімум ключів у будь-якому вузлі (крім кореня)
max_depth	3	Дерево не може мати більше 3 рівнів
KEY_CHARS	4	Кількість символів для order-preserving prefix
BASE	27	Основа полінома (26 літер + 0 для padding)

3. Приклади розміщення хеш-кодів у B+ дереві

3.1 Хеш-коди всіх записів довідника

Таблиця 3 – Хеш-коди всіх записів

Ім’я	Prefix (name_to_key)	Full key (name_to_key_full)	Позиція в дереві
Anna	30 268	16 085 656 188	Листок 1
Annabelle	30 268	16 085 743 758	Листок 1
Boris	50 796	26 995 077 151	Листок 1
Charlie	64 926	34 504 445 510	Листок 2
David	80 064	42 549 292 324	Листок 2
Egor	103 941	55 238 508 981	Листок 3
Fiona	125 078	66 471 577 495	Листок 3
George	141 849	75 384 377 703	Листок 4
Helen	161 438	85 794 772 268	Листок 4
Igor	182 673	97 079 921 793	Листок 5
John	207 995	110 537 070 795	Листок 5
Kate	217 787	115 740 941 067	Листок 6
Leo	240 246	127 676 574 486	Листок 6
Mia	262 467	139 485 724 947	Листок 6

3.2 Код для виведення структури дерева

Код для виведення структури:

```
def print_tree(self):
        print(f"\nB+ Tree  order={self.order}, "
              f"depth={self._depth}/{self.max_depth}")
        if not self.root.keys and not self.root.children:
            print("Empty tree")
            return

        queue = [(self.root, 0)]
        cur_level, line = 0, []
        while queue:
            node, level = queue.pop(0)
            if level != cur_level:
                print(f"Level {cur_level}: " + "   ".join(line))
                line, cur_level = [], level
            if node.is_leaf:
                names = [c[0] for c in node.children]
                line.append(f"[{', '.join(names)}]")
            else:
                line.append(f"({', '.join(str(k) for k in node.keys)})")
                for child in node.children:
                    queue.append((child, level + 1))
        if line:
            print(f"Level {cur_level}: " + "   ".join(line))

        node  = self.root
        while not node.is_leaf:
            node = node.children[0]
        chain = []
        while node:
            chain.append([c[0] for c in node.children])
            node = node.next
        print("Linked chain: " + " => ".join(str(b) for b in chain))
        print()
```

3.3 Структура розміщення хеш-кодів у B+-дереві

Вивід структури за допомогою коду, якщо додавати записи послідовно з табл. 1:

B+ Tree order=4, depth=3/3
Level 0: (75384377703)
Level 1: (34504445510, 55238508981) (97079921793, 115740941067)
Level 2: [Anna, Annabelle, Boris] [Charlie, David] [Egor, Fiona] [George, Helen] [Igor, John] [Kate, Leo, Mia]
Linked chain: ['Anna', 'Annabelle', 'Boris'] => ['Charlie', 'David'] => ['Egor', 'Fiona'] => ['George', 'Helen'] => ['Igor', 'John'] => ['Kate', 'Leo', 'Mia']

Точна структура B+-дерева наведена на рис.1.

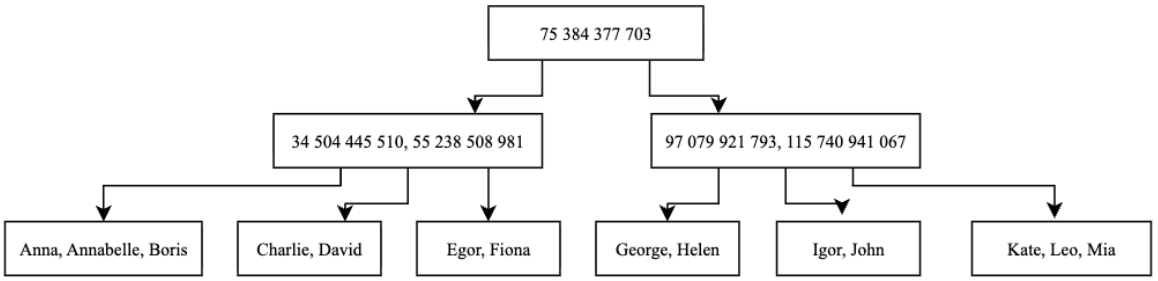


Рис. 1 – структура B+-дерева

4. Схеми пошуку значень в дереві

4.1 Точний пошук – search()

```
def search(self, name: str):
        key = self._hash_fn(name)

        if self._is_key_in_cached_leaf(key):
            self._cache_hits += 1
            leaf = self._cached_leaf
            for i, k in enumerate(leaf.keys):
                if k == key:
                    return leaf.children[i]
            return None

        self._cache_misses += 1
        leaf = self._find_leaf(key)
        self._cached_leaf = leaf

        for i, k in enumerate(leaf.keys):
            if k == key:
                return leaf.children[i]
        return None
```

Приклад 1:
search('David')
key = name_to_key_full('David') = 42549292324
Крок 1 – корінь: 42549292324 < 75384377703 => іти вліво
Крок 2 – вузол: 42549292324 > 34504445510 => i=1
42549292324 < 55238508981 => зупинились, i=1
йдемо у children[1] = листок [Charlie, David]
Крок 3 – у листку знаходимо ключ David => повертаємо ('David', '380-95-123-45-70')

Приклад 2:
search('Zara')
key = name_to_key_full('Zara') = 142737428574

Крок 1 – корінь: 142737428574 > 75384377703 => іти у children[1] (правий вузол)
Крок 2 – вузол: 142737428574 > 97079921793 => i = 1
142737428574 > 115740941067 => i = 2
іти у children[2]
Крок 3 – Листок: [Kate, Leo, Mia]
Перебираємо всі ключі – жоден не збігається => return None

4.2 Пошук з кешем листка

Дерево запам'ятовує останній використаний листок. При наступному запиті – спочатку перевіряє чи ключ потрапляє у діапазон цього листка, і якщо так – повертає результат без спуску по дереву.

```
def _is_key_in_cached_leaf(self, key) -> bool:
        leaf = self._cached_leaf
        if leaf is None or not leaf.keys:
            return False
        return leaf.keys[0] <= key <= leaf.keys[-1]
```

Приклад виводу програми:

Search('Annabelle') => Annabelle [Cache not used] – перший запит
Search('Boris') => Boris [Cache used] – знаходиться у листку з Annabelle

При будь-якій вставці або видаленні викликається _invalidate_cache() — бо структура листків могла змінитись (split, merge, borrow):

```
def _invalidate_cache(self):
        self._cached_leaf = None
```

4.3 Порівняння операцій пошуку

У табл. 4 наведено порівняння схем пошуку значень в дереві.

Таблиця 4 – Порівняння операцій пошуку

Операція	Початок	Напрям руху	Умова зупинки	Складність	Кеш
search(name)	Корінь	Вниз по дереву, потім у листку лінійно	Знайшов ключ або перебрав усі у листку	O(log n)	Якщо ключ у діапазоні кешованого листка, спуск пропуска-ється
search_greater (name)	Корінь	Вниз до стартово-го листка, потім вправо по .next	.next = None (кінець ланцюжка)	O(log n + k)	Стартовий листок береться з кешу якщо підходить
search_less (name)	Найлівіший листок (children[0] від кореня)	Вправо по .next від початку	.next = None (до заданого значення)	O(n) worst case	Завжди стартує з першого листка

4.4 Порівняння операцій вставки та видалення

Таблиця 5 – Порівняння операцій вставки та видалення

	Напрям	Зміна дерева	Висота дерева	Кеш після операції	Складність
Вставка	Вниз, потім вгору (split)	Split: листок або внутрішній вузол	Може зрости на 1	Скидається	O(log n) + split
Видалення	Вниз, потім вгору (borrow)	Merge або borrow	Може зменшитись на 1	Скидається	O(log n) + merge/ borrow

Додаток А. Код програми

```
KEY_CHARS = 4
BASE = 27

def name_to_key(name: str) -> int:
    chars = name.lower()[:KEY_CHARS]
    key = 0
    for ch in chars:
        if 'a' <= ch <= 'z':
            code = ord(ch) - ord('a') + 1
        else:
            code = 0
        key = key * BASE + code

    padding = KEY_CHARS - len(chars)
    key *= BASE ** padding

    return key

def name_to_key_full(name: str) -> int:
    prefix = name_to_key(name)

    suffix_chars = name.lower()[KEY_CHARS:]
    rolling = 0
    for ch in suffix_chars:
        rolling = rolling * 31 + ord(ch)

    SUFFIX_MOD = BASE ** KEY_CHARS
    return prefix * SUFFIX_MOD + (rolling % SUFFIX_MOD)

class BPlusNode:
    def __init__(self, is_leaf=False):
        self.is_leaf = is_leaf
        self.keys = []
        self.children = []
        self.parent = None
        self.next = None

    def __repr__(self):
        return f"{'Leaf' if self.is_leaf else 'Node'}{self.keys}"

class BPlusTree:
    def __init__(self, order=4, max_depth=3, use_full_hash=True):
        self.root          = BPlusNode(is_leaf=True)
        self.order         = order
        self.min_keys      = (order + 1) // 2
        self.max_depth     = max_depth
        self._depth        = 1
        self._hash_fn      = name_to_key_full if use_full_hash else name_to_key

        max_capacity = (order - 1) * (order ** (max_depth - 1))
        print(f"\n[BPlusTree] order={order}, max_depth={max_depth}, "
              f"min_keys={self.min_keys}, max_capacity≈{max_capacity} records")

        self._cached_leaf  = None
        self._cache_hits   = 0
        self._cache_misses = 0


    def _check_depth_before_split(self):
        if self._depth + 1 > self.max_depth:
            raise OverflowError(
                f"\n  Дерево не може рости далі: поточна глибина {self._depth}, "
                f"ліміт {self.max_depth}.\n"
                f"  Рішення: збільшіть order (зараз {self.order}) або max_depth.\n"
```

```
                f"  При order={self.order} і max_depth={self.max_depth} "
                f"максимум ≈ {(self.order-1) * self.order**(self.max_depth-1)}
записів."
            )


    def _find_leaf(self, key):
        node = self.root
        while not node.is_leaf:
            i = 0
            while i < len(node.keys) and key >= node.keys[i]:
                i += 1
            node = node.children[i]
        return node


    def insert(self, name: str, phone: str):
        self._invalidate_cache()
        key  = self._hash_fn(name)
        leaf = self._find_leaf(key)

        idx = 0
        while idx < len(leaf.keys) and leaf.keys[idx] < key:
            idx += 1

        leaf.keys.insert(idx, key)
        leaf.children.insert(idx, (name, phone))

        if len(leaf.keys) >= self.order:
            self._split_leaf(leaf)

    def _split_leaf(self, leaf):
        new_leaf = BPlusNode(is_leaf=True)
        mid = len(leaf.keys) // 2

        new_leaf.keys     = leaf.keys[mid:]
        new_leaf.children = leaf.children[mid:]
        leaf.keys         = leaf.keys[:mid]
        leaf.children     = leaf.children[:mid]

        new_leaf.next = leaf.next
        leaf.next     = new_leaf

        if leaf is self.root:
            self._check_depth_before_split()
            new_root = BPlusNode(is_leaf=False)
            new_root.keys     = [new_leaf.keys[0]]
            new_root.children = [leaf, new_leaf]
            leaf.parent       = new_root
            new_leaf.parent   = new_root
            self.root         = new_root
            self._depth      += 1
        else:
            self._insert_into_parent(leaf, new_leaf.keys[0], new_leaf)

    def _insert_into_parent(self, left, key, right):
        parent = left.parent
        idx    = parent.children.index(left)
        parent.keys.insert(idx, key)
        parent.children.insert(idx + 1, right)
        right.parent = parent

        if len(parent.keys) >= self.order:
            self._split_internal(parent)

    def _split_internal(self, node):
        new_node     = BPlusNode(is_leaf=False)
        mid          = len(node.keys) // 2
```

```
        up_key       = node.keys[mid]

        new_node.keys     = node.keys[mid + 1:]
        new_node.children = node.children[mid + 1:]
        for child in new_node.children:
            child.parent = new_node

        node.keys     = node.keys[:mid]
        node.children = node.children[:mid + 1]

        if node is self.root:
            self._check_depth_before_split()
            new_root = BPlusNode(is_leaf=False)
            new_root.keys     = [up_key]
            new_root.children = [node, new_node]
            node.parent       = new_root
            new_node.parent   = new_root
            self.root         = new_root
            self._depth      += 1
        else:
            self._insert_into_parent(node, up_key, new_node)


    def _is_key_in_cached_leaf(self, key) -> bool:
        leaf = self._cached_leaf
        if leaf is None or not leaf.keys:
            return False
        return leaf.keys[0] <= key <= leaf.keys[-1]

    def _invalidate_cache(self):
        self._cached_leaf = None


    def search(self, name: str):
        key = self._hash_fn(name)

        if self._is_key_in_cached_leaf(key):
            self._cache_hits += 1
            leaf = self._cached_leaf
            for i, k in enumerate(leaf.keys):
                if k == key:
                    return leaf.children[i]
            return None

        self._cache_misses += 1
        leaf = self._find_leaf(key)
        self._cached_leaf = leaf

        for i, k in enumerate(leaf.keys):
            if k == key:
                return leaf.children[i]
        return None

    def search_greater(self, name: str):
        key = self._hash_fn(name)

        if self._is_key_in_cached_leaf(key):
            self._cache_hits += 1
            leaf = self._cached_leaf
        else:
            self._cache_misses += 1
            leaf = self._find_leaf(key)
            self._cached_leaf = leaf

        result = []
        while leaf:
            for i, k in enumerate(leaf.keys):
                if k > key:
```

```
                    result.append(leaf.children[i])
            leaf = leaf.next
        return result

    def search_less(self, name: str):
        key  = self._hash_fn(name)
        node = self.root
        while not node.is_leaf:
            node = node.children[0]
        result = []
        while node:
            for i, k in enumerate(node.keys):
                if k < key:
                    result.append(node.children[i])
            node = node.next
        return result


    def delete(self, name: str) -> bool:
        self._invalidate_cache()
        key  = self._hash_fn(name)
        leaf = self._find_leaf(key)

        if key not in leaf.keys:
            return False

        idx = leaf.keys.index(key)
        leaf.keys.pop(idx)
        leaf.children.pop(idx)

        if leaf is self.root:
            return True

        if len(leaf.keys) >= self.min_keys:
            if idx == 0 and leaf.keys:
                self._update_parent_key(leaf)
            return True

        if self._borrow_from_sibling(leaf):
            return True

        self._merge_leaf(leaf)
        return True

    def _update_parent_key(self, leaf):
        if not leaf.parent or not leaf.keys:
            return
        parent = leaf.parent
        try:
            child_idx = parent.children.index(leaf)
        except ValueError:
            return
        if child_idx > 0:
            parent.keys[child_idx - 1] = leaf.keys[0]

    def _get_siblings(self, node):
        parent = node.parent
        idx    = parent.children.index(node)
        left   = parent.children[idx - 1] if idx > 0 else None
        right  = parent.children[idx + 1] if idx < len(parent.children) - 1 else
None
        return left, right, idx

    def _borrow_from_sibling(self, leaf) -> bool:
        parent         = leaf.parent
        left, right, idx = self._get_siblings(leaf)

        if right and len(right.keys) > self.min_keys:
```

```
            leaf.keys.append(right.keys.pop(0))
            leaf.children.append(right.children.pop(0))
            if right.keys:
                parent.keys[idx] = right.keys[0]
            return True

        if left and len(left.keys) > self.min_keys:
            leaf.keys.insert(0, left.keys.pop(-1))
            leaf.children.insert(0, left.children.pop(-1))
            parent.keys[idx - 1] = leaf.keys[0]
            return True

        return False

    def _merge_leaf(self, leaf):
        parent           = leaf.parent
        left, right, idx = self._get_siblings(leaf)

        if right:
            leaf.keys.extend(right.keys)
            leaf.children.extend(right.children)
            leaf.next = right.next
            parent.keys.pop(idx)
            parent.children.pop(idx + 1)
        else:
            left.keys.extend(leaf.keys)
            left.children.extend(leaf.children)
            left.next = leaf.next
            parent.keys.pop(idx - 1)
            parent.children.pop(idx)

        if parent is self.root:
            if not parent.keys:
                self.root        = parent.children[0]
                self.root.parent = None
                self._depth     -= 1
        elif len(parent.keys) < self.min_keys:
            self._fix_internal(parent)

    def _fix_internal(self, node):
        parent           = node.parent
        left, right, idx = self._get_siblings(node)

        if right and len(right.keys) > self.min_keys:
            node.keys.append(parent.keys[idx])
            parent.keys[idx] = right.keys.pop(0)
            child = right.children.pop(0)
            child.parent = node
            node.children.append(child)
            return

        if left and len(left.keys) > self.min_keys:
            node.keys.insert(0, parent.keys[idx - 1])
            parent.keys[idx - 1] = left.keys.pop(-1)
            child = left.children.pop(-1)
            child.parent = node
            node.children.insert(0, child)
            return

        if right:
            node.keys.append(parent.keys.pop(idx))
            node.keys.extend(right.keys)
            for child in right.children:
                child.parent = node
            node.children.extend(right.children)
            parent.children.pop(idx + 1)
        else:
            left.keys.append(parent.keys.pop(idx - 1))
```

```
            left.keys.extend(node.keys)
            for child in node.children:
                child.parent = left
            left.children.extend(node.children)
            parent.children.pop(idx)
            node = left

        if parent is self.root:
            if not parent.keys:
                self.root        = parent.children[0]
                self.root.parent = None
                self._depth     -= 1
        elif len(parent.keys) < self.min_keys:
            self._fix_internal(parent)


    def print_tree(self):
        print(f"\nB+ Tree  order={self.order}, "
              f"depth={self._depth}/{self.max_depth}")
        if not self.root.keys and not self.root.children:
            print("Empty tree")
            return

        queue = [(self.root, 0)]
        cur_level, line = 0, []
        while queue:
            node, level = queue.pop(0)
            if level != cur_level:
                print(f"Level {cur_level}: " + "   ".join(line))
                line, cur_level = [], level
            if node.is_leaf:
                names = [c[0] for c in node.children]
                line.append(f"[{', '.join(names)}]")
            else:
                line.append(f"({', '.join(str(k) for k in node.keys)})")
                for child in node.children:
                    queue.append((child, level + 1))
        if line:
            print(f"Level {cur_level}: " + "   ".join(line))

        node  = self.root
        while not node.is_leaf:
            node = node.children[0]
        chain = []
        while node:
            chain.append([c[0] for c in node.children])
            node = node.next
        print("Linked chain: " + " => ".join(str(b) for b in chain))
        print()


if __name__ == "__main__":
    print("\nHash functions:")
    names_test = ["Anna", "Annabelle", "Boris", "Charlie", "David", "Egor",
                  "Fiona", "George", "Helen", "Igor", "John", "Kate", "Leo", "Mia"]
    print(f"{'name':<12} {'rolling hash':>16}  {'order-preserving':>18}
{'combined':>20}")
    print("-" * 72)

    def rolling(name):
        k = 0
        for ch in name.lower(): k = k * 31 + ord(ch)
        return k

    for n in names_test:
        print(f"{n:<12} {rolling(n):>16}  {name_to_key(n):>18}
{name_to_key_full(n):>20}")
```

```
tree = BPlusTree(order=4, max_depth=3, use_full_hash=True)

records = [
    ("Boris",   "380-95-123-45-68"),
    ("Charlie", "380-95-123-45-69"),
    ("David",   "380-95-123-45-70"),
    ("Egor",    "380-95-123-45-71"),
    ("Annabelle", "380-95-123-45-72"),
    ("Anna",    "380-95-123-45-72"),
    ("Fiona",   "380-95-123-45-73"),
    ("George",  "380-95-123-45-74"),
    ("Helen",   "380-95-123-45-75"),
    ("Igor",    "380-95-123-45-76"),
    ("John",    "380-95-123-45-77"),
    ("Kate",    "380-95-123-45-78"),
    ("Leo",     "380-95-123-45-79"),
    ("Mia",     "380-95-123-45-80"),
]
for name, phone in records:
    tree.insert(name, phone)

print("\nTree after insertions:")
tree.print_tree()

print("\nSearches:")
for name in ["Anna", "David", "Zara"]:
    result = tree.search(name)
    print(f"Search('{name}') => {result}")

print("\nSearch after Fiona")
for r in tree.search_greater("Fiona"):
    print(f"{r[0]}")

print("\nSearch before David")
for r in tree.search_less("David"):
    print(f"{r[0]}")

print("\nDeletions:")
for name in ["Anna", "Egor", "Charlie", "Peter", "Helen", "Fiona"]:
    ok = tree.delete(name)
    print(f"Delete('{name}') => {'OK' if ok else 'Not found'}")
tree.print_tree()

def search_verbose(tree, name):
    hits_before = tree._cache_hits
    result = tree.search(name)
    hit = tree._cache_hits > hits_before
    status = "Cache used" if hit else "Cache not used"
    name_str = result[0] if result else "None"
    print(f"Search('{name}') => {name_str:<12}  [{status}]")

search_verbose(tree, "Annabelle")
search_verbose(tree, "Boris")
search_verbose(tree, "Charlie")
search_verbose(tree, "David")
search_verbose(tree, "George")
search_verbose(tree, "Helen")
```

Приклад виконання програми:

Hash functions:

name	rolling hash	order-preserving	combined
Anna	2998944	30268	16085656188
Annabelle	85857314237730	30268	16085743758
Boris	93924783	50796	26995077151
Charlie	90933380978	64926	34504445510
David	95358580	80064	42549292324
Egor	3111429	103941	55238508981
Fiona	97437375	125078	66471577495
George	3045497823	141849	75384377703
Helen	99162104	161438	85794772268
Igor	3230593	182673	97079921793
John	3267851	207995	110537070795
Kate	3284551	217787	115740941067
Leo	107030	240246	127676574486
Mia	108101	262467	139485724947

[BPlusTree] order=4, max_depth=3, min_keys=2, max_capacity≈48 records

Tree after insertions:

B+ Tree order=4, depth=3/3
Level 0: (66471577495)
Level 1: (26995077151, 42549292324) (85794772268, 110537070795, 127676574486)
Level 2: [Anna, Annabelle] [Boris, Charlie] [David, Egor] [Fiona, George] [Helen, Igor] [John, Kate] [Leo, Mia]
Linked chain: ['Anna', 'Annabelle'] => ['Boris', 'Charlie'] => ['David', 'Egor'] => ['Fiona', 'George'] => ['Helen', 'Igor'] => ['John', 'Kate'] => ['Leo', 'Mia']

Searches:
Search('Anna') => ('Anna', '380-95-123-45-72')
Search('David') => ('David', '380-95-123-45-70')

Search('Zara') => None

Search after Fiona
George
Helen
Igor
John
Kate
Leo
Mia

Search before David
Anna
Annabelle
Boris
Charlie

Deletions:
Delete('Anna') => OK
Delete('Egor') => OK
Delete('Charlie') => OK
Delete('Peter') => Not found
Delete('Helen') => OK
Delete('Fiona') => OK

B+ Tree order=4, depth=2/3
Level 0: (34504445510, 110537070795, 127676574486)
Level 1: [Annabelle, Boris] [David, George, Igor] [John, Kate] [Leo, Mia]
Linked chain: ['Annabelle', 'Boris'] => ['David', 'George', 'Igor'] => ['John', 'Kate'] => ['Leo', 'Mia']

Search('Annabelle') => Annabelle [Cache not used]
Search('Boris') => Boris [Cache used]
Search('Charlie') => None [Cache not used]
Search('David') => David [Cache used]
Search('George') => George [Cache used]
Search('Helen') => None [Cache used]